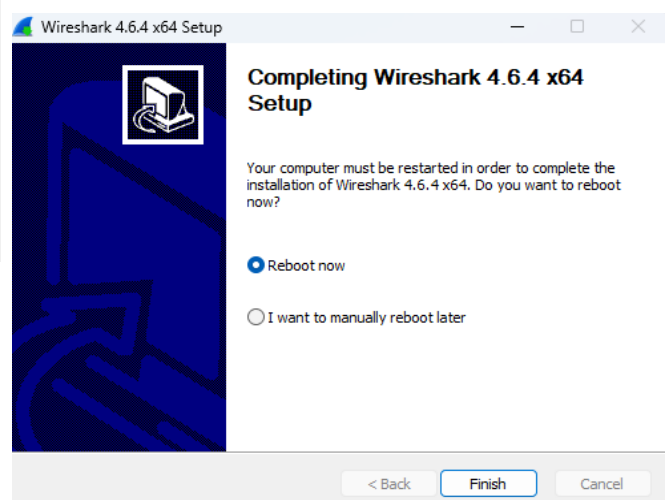
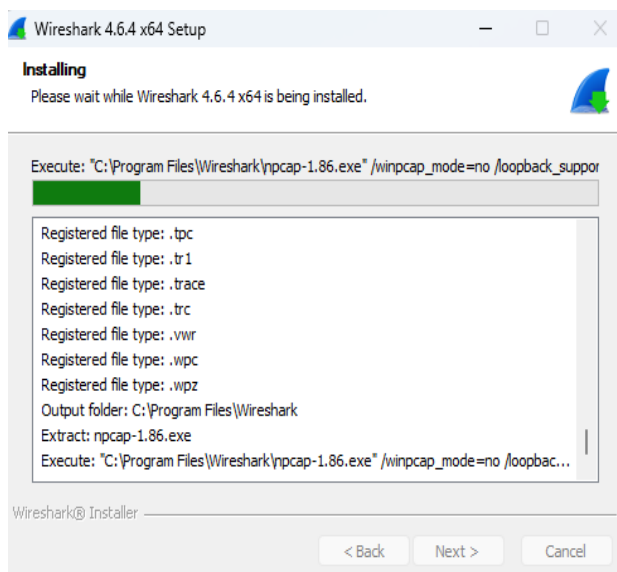
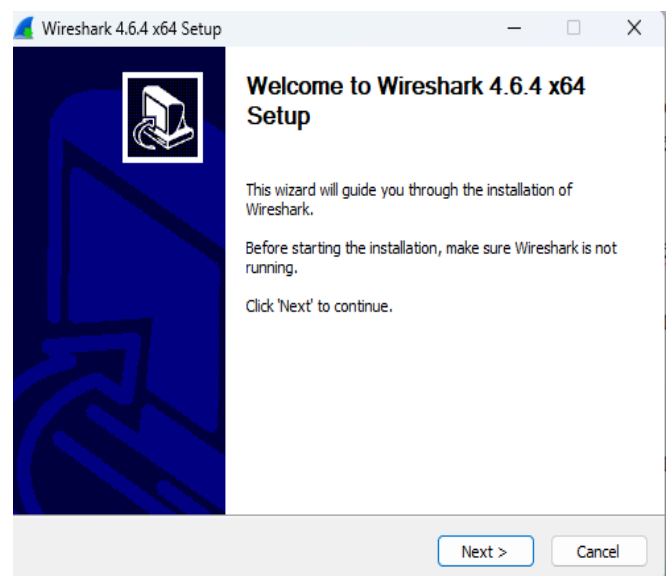
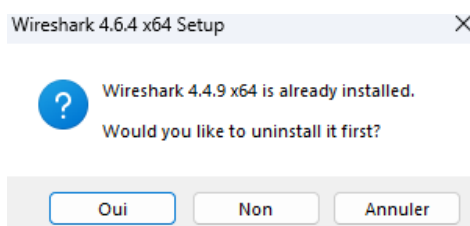
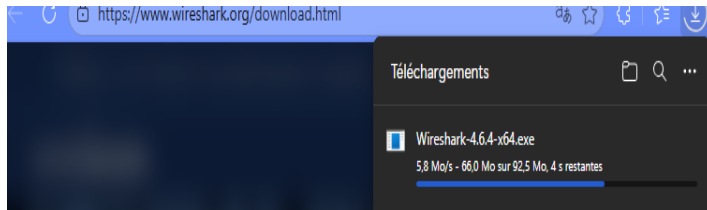
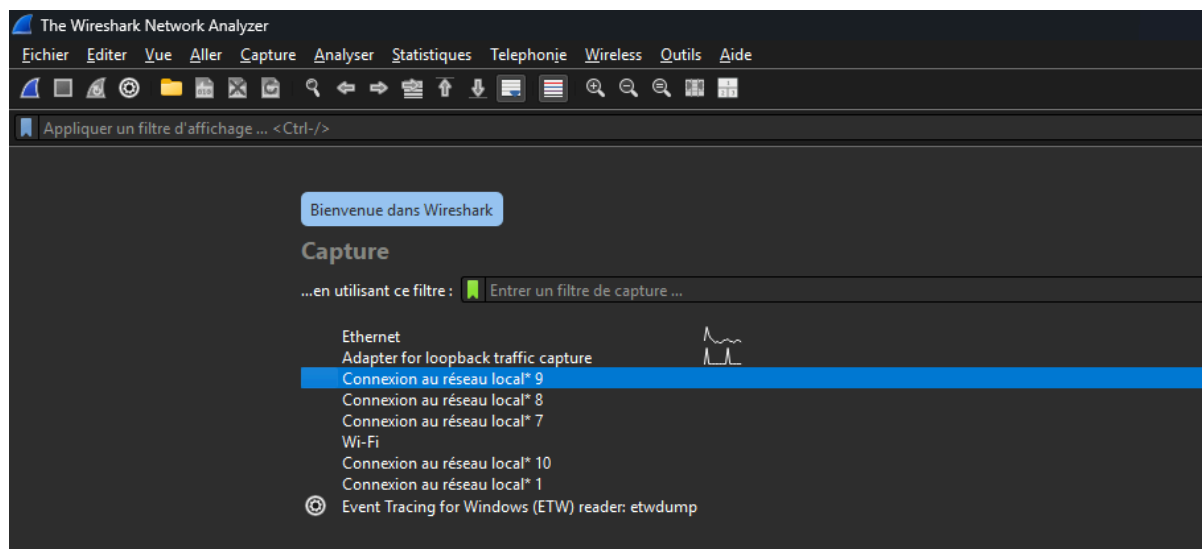


TP 3 fronte avance

Partie 1: Installation de wireshark





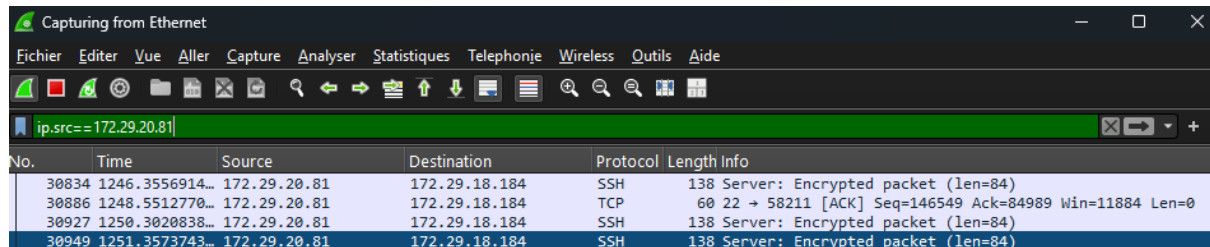
1) Tester les boutons suivants et dites à quoi ils servent :

Triangle bleu	c'est wireshark lui même
Carré rouge	arrêter la capture de paquet
Triangle vert avec flèche blanche	redémarrer la capture en cours
Triangle vert sans la flèche	démarrer la capture de paquet
Roue crampton	option de capture

Partie 2 : faire un filtre de trame wireshark

2) Tester les filtres suivants et dites à quoi ils servent

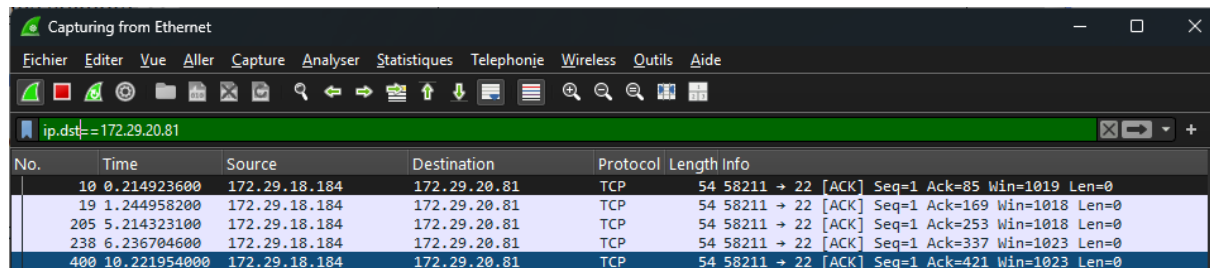
ip.src permet d'afficher uniquement les paquets dont l'adresse IP source est celle indiquée.



The screenshot shows the Wireshark interface with the filter `ip.src == 172.29.20.81` applied. The packet list shows four packets from source 172.29.20.81 to destination 172.29.18.184. The first and third packets are SSH encrypted packets, while the second and fourth are TCP ACKs.

No.	Time	Source	Destination	Protocol	Length	Info
30834	1246.3556914...	172.29.20.81	172.29.18.184	SSH	138	Server: Encrypted packet (len=84)
30886	1248.5512770...	172.29.20.81	172.29.18.184	TCP	60	22 → 58211 [ACK] Seq=146549 Ack=84989 Win=11884 Len=0
30927	1250.3020838...	172.29.20.81	172.29.18.184	SSH	138	Server: Encrypted packet (len=84)
30949	1251.3573743...	172.29.20.81	172.29.18.184	SSH	138	Server: Encrypted packet (len=84)

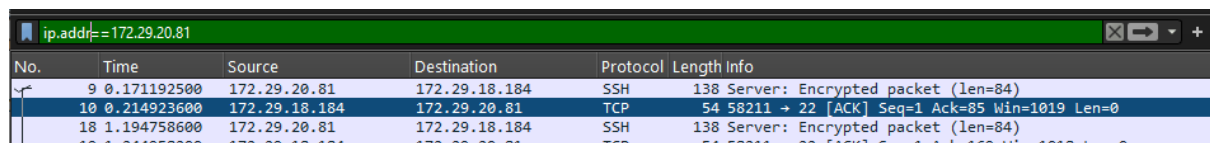
ip.dst permet d'afficher uniquement les paquets dont l'adresse IP de destination correspond à celle indiquée.



The screenshot shows the Wireshark interface with the filter `ip.dst == 172.29.20.81` applied. The packet list shows five packets from source 172.29.18.184 to destination 172.29.20.81. All packets are TCP ACKs.

No.	Time	Source	Destination	Protocol	Length	Info
10	0.214923600	172.29.18.184	172.29.20.81	TCP	54	58211 → 22 [ACK] Seq=1 Ack=85 Win=1019 Len=0
19	1.244958200	172.29.18.184	172.29.20.81	TCP	54	58211 → 22 [ACK] Seq=1 Ack=169 Win=1018 Len=0
205	5.214323100	172.29.18.184	172.29.20.81	TCP	54	58211 → 22 [ACK] Seq=1 Ack=253 Win=1018 Len=0
238	6.236704600	172.29.18.184	172.29.20.81	TCP	54	58211 → 22 [ACK] Seq=1 Ack=337 Win=1023 Len=0
400	10.221954000	172.29.18.184	172.29.20.81	TCP	54	58211 → 22 [ACK] Seq=1 Ack=421 Win=1023 Len=0

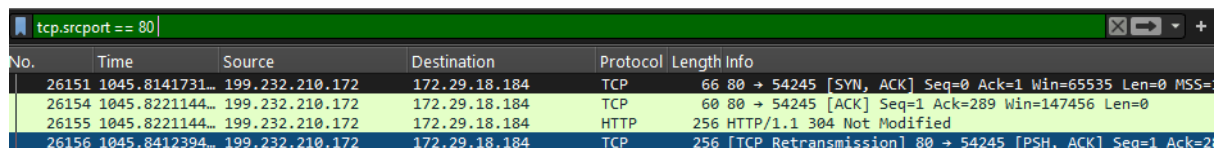
ip.addr permet d'afficher tous les paquets où l'adresse IP apparaît, que ce soit en source ou en destination.



The screenshot shows the Wireshark interface with the filter `ip.addr == 172.29.20.81` applied. The packet list shows five packets involving 172.29.20.81. The first and third packets are SSH encrypted packets, while the second and fourth are TCP ACKs. The fifth packet is a TCP retransmission.

No.	Time	Source	Destination	Protocol	Length	Info
9	0.171192500	172.29.20.81	172.29.18.184	SSH	138	Server: Encrypted packet (len=84)
10	0.214923600	172.29.18.184	172.29.20.81	TCP	54	58211 → 22 [ACK] Seq=1 Ack=85 Win=1019 Len=0
18	1.194758600	172.29.20.81	172.29.18.184	SSH	138	Server: Encrypted packet (len=84)
19	1.244958200	172.29.18.184	172.29.20.81	TCP	54	58211 → 22 [ACK] Seq=1 Ack=169 Win=1018 Len=0
205	5.214323100	172.29.18.184	172.29.20.81	TCP	54	58211 → 22 [ACK] Seq=1 Ack=253 Win=1018 Len=0

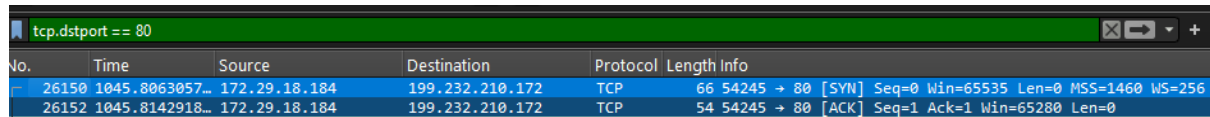
tcp.srcport permet de montrer les paquets TCP dont le port source est celui indiqué.



The screenshot shows the Wireshark interface with the filter `tcp.srcport == 80` applied. The packet list shows four packets from source 199.232.210.172 to destination 172.29.18.184. The first two are TCP SYN and ACK, the third is an HTTP 304 Not Modified response, and the fourth is a TCP retransmission of the SYN packet.

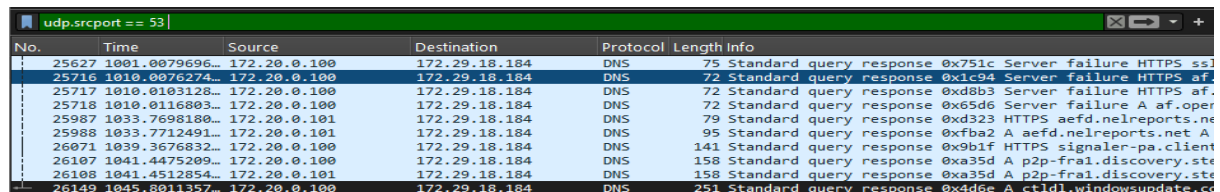
No.	Time	Source	Destination	Protocol	Length	Info
26151	1045.8141731...	199.232.210.172	172.29.18.184	TCP	66	80 → 54245 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=
26154	1045.8221144...	199.232.210.172	172.29.18.184	TCP	60	80 → 54245 [ACK] Seq=1 Ack=289 Win=147456 Len=0
26155	1045.8221144...	199.232.210.172	172.29.18.184	HTTP	256	HTTP/1.1 304 Not Modified
26156	1045.8412394...	199.232.210.172	172.29.18.184	TCP	256	[TCP Retransmission] 80 → 54245 [PSH, ACK] Seq=1 Ack=2

tcp.dstport permet de montrer les paquets TCP dont le **port de destination** est celui indiqué.



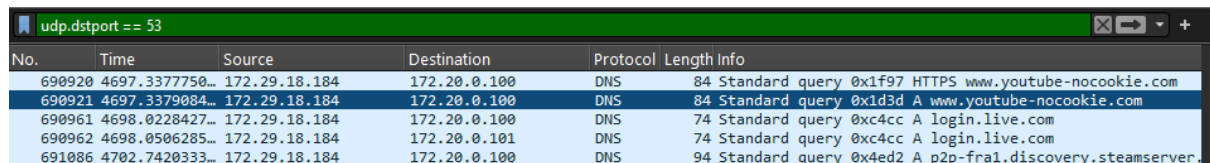
No.	Time	Source	Destination	Protocol	Length	Info
26150	1045.8063057...	172.29.18.184	199.232.210.172	TCP	66	54245 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256
26152	1045.8142918...	172.29.18.184	199.232.210.172	TCP	54	54245 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0

udp.srcport permet d'afficher les paquets UDP dont le **port source** correspond à celui indiqué.



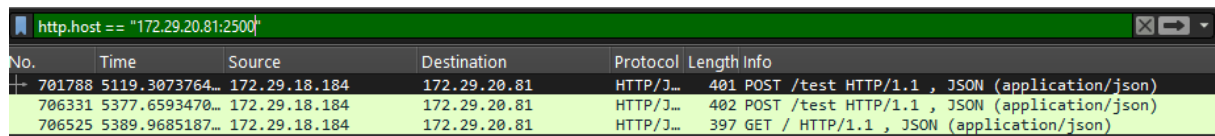
No.	Time	Source	Destination	Protocol	Length	Info
25627	1001.0079696...	172.20.0.100	172.29.18.184	DNS	75	Standard query response 0x751c Server failure HTTPS ss
25716	1010.0076274...	172.20.0.100	172.29.18.184	DNS	72	Standard query response 0xc194 Server failure HTTPS af
25717	1010.0103128...	172.20.0.100	172.29.18.184	DNS	72	Standard query response 0xd8b3 Server failure HTTPS af
25718	1010.0116803...	172.20.0.100	172.29.18.184	DNS	72	Standard query response 0x65d6 Server failure A af.oper
25987	1033.7698180...	172.20.0.101	172.29.18.184	DNS	79	Standard query response 0xd323 HTTPS aefd.nelreports.ne
25988	1033.7712491...	172.20.0.101	172.29.18.184	DNS	95	Standard query response 0xfba2 A aefd.nelreports.net A
26071	1039.3676832...	172.20.0.100	172.29.18.184	DNS	141	Standard query response 0x9b1f HTTPS signaler-pa.client
26107	1041.4475209...	172.20.0.100	172.29.18.184	DNS	158	Standard query response 0xa35d A p2p-fra1.discovery.ste
26108	1041.4512854...	172.20.0.101	172.29.18.184	DNS	158	Standard query response 0xa35d A p2p-fra1.discovery.ste
26149	1045.8011357...	172.20.0.100	172.29.18.184	DNS	251	Standard query response 0x4d6e A ctld1.windowsupdate.co

udp.dstport permet d'afficher les paquets UDP dont le **port de destination** correspond à celui indiqué.



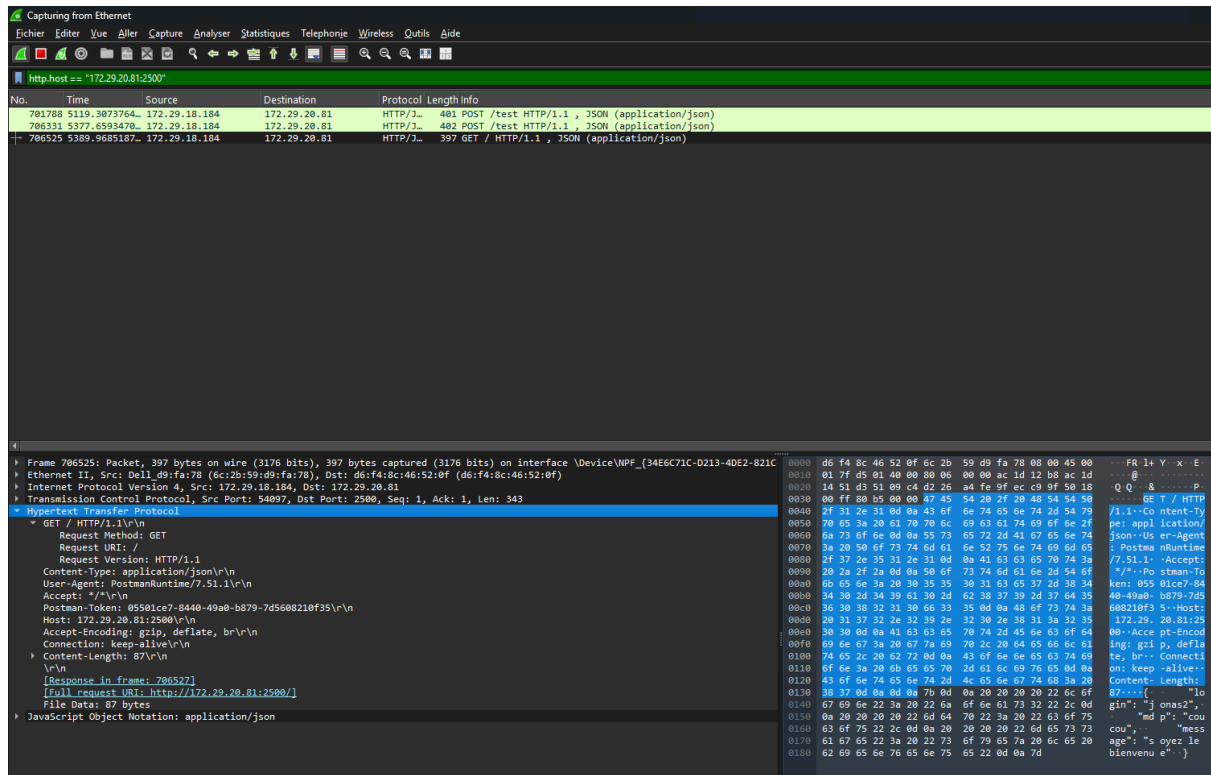
No.	Time	Source	Destination	Protocol	Length	Info
690920	4697.3377750...	172.29.18.184	172.20.0.100	DNS	84	Standard query 0x1f97 HTTPS www.youtube-nocookie.com
690921	4697.3379084...	172.29.18.184	172.20.0.100	DNS	84	Standard query 0x1d3d A www.youtube-nocookie.com
690961	4698.0228427...	172.29.18.184	172.20.0.100	DNS	74	Standard query 0xc4cc A login.live.com
690962	4698.0506285...	172.29.18.184	172.20.0.101	DNS	74	Standard query 0xc4cc A login.live.com
691086	4702.7420333...	172.29.18.184	172.20.0.100	DNS	94	Standard query 0x4ed2 A p2p-fra1.discovery.steamserver.

http.host permet de filtrer les paquets HTTP dont le **champ Host** correspond exactement à la valeur donnée.



No.	Time	Source	Destination	Protocol	Length	Info
701788	5119.3073764...	172.29.18.184	172.29.20.81	HTTP/1...	401	POST /test HTTP/1.1 , JSON (application/json)
706331	5377.6593470...	172.29.18.184	172.29.20.81	HTTP/J...	402	POST /test HTTP/1.1 , JSON (application/json)
706525	5389.9685187...	172.29.18.184	172.29.20.81	HTTP/J...	397	GET / HTTP/1.1 , JSON (application/json)

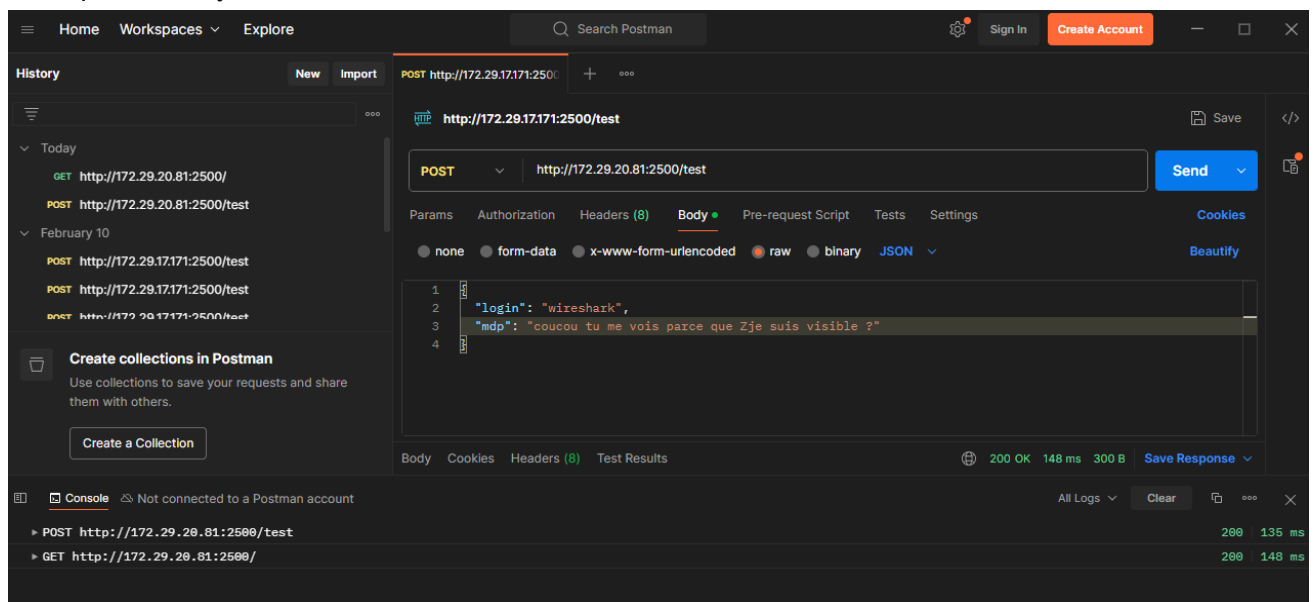
Voilà la trame



Partie 3 : analyse d'une requête http POST

3) Pour valider ce TP vous devez capturer une trame http entre votre application server et votre client postman. Au format POST. Vous enverrez le json suivant :

La requête envoyée



la trame frame du login

The image shows a Wireshark packet capture of a frame. The left pane displays the packet details, and the right pane shows the raw packet data in hexadecimal and ASCII. The packet is a response in frame 3031, with a full request URI of http://172.29.20.81:2500/test. The data is a JavaScript Object Notation (JSON) object with a content length of 89 bytes. The JSON object has two members: 'login' and 'mdp'. The 'login' member has a path of /login and a value of wireshark. The 'mdp' member has a path of /mdp and a value of coucou tu me vois parce que Zje suis vis. The raw data shows the JSON object as {"login": "wireshark", "mdp": "coucou tu me vois parce que Zje suis vis"}. The 'login' member is highlighted in blue in the details pane.

la trame du mdp

The image shows a Wireshark packet capture of a frame, similar to the one above. The left pane displays the packet details, and the right pane shows the raw packet data in hexadecimal and ASCII. The packet is a response in frame 3031, with a full request URI of http://172.29.20.81:2500/test. The data is a JavaScript Object Notation (JSON) object with a content length of 89 bytes. The JSON object has two members: 'login' and 'mdp'. The 'login' member has a path of /login and a value of wireshark. The 'mdp' member has a path of /mdp and a value of coucou tu me vois parce que Zje suis vis. The raw data shows the JSON object as {"login": "wireshark", "mdp": "coucou tu me vois parce que Zje suis vis"}. The 'mdp' member is highlighted in blue in the details pane.

4) Quelle est la valeur de la propriété de l'entête http suivante : Content-Length

The image shows a Wireshark packet capture of a frame. The left pane displays the packet details, and the right pane shows the raw packet data in hexadecimal and ASCII. The packet is a response in frame 3031, with a full request URI of http://172.29.20.81:2500/test. The data is a JavaScript Object Notation (JSON) object with a content length of 89 bytes. The JSON object has two members: 'login' and 'mdp'. The 'login' member has a path of /login and a value of wireshark. The 'mdp' member has a path of /mdp and a value of coucou tu me vois parce que Zje suis vis. The raw data shows the JSON object as {"login": "wireshark", "mdp": "coucou tu me vois parce que Zje suis vis"}. The 'Content-Length' header is highlighted in blue in the details pane, showing a value of 89.

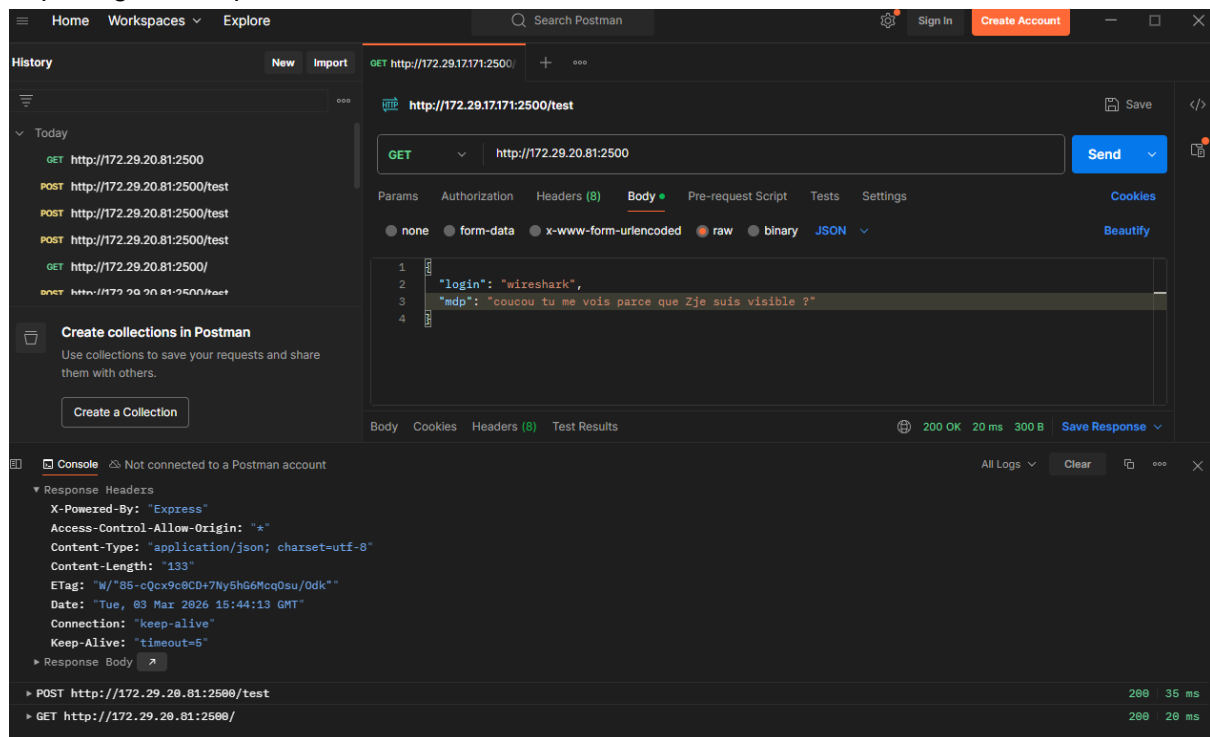
5) Est-ce que le mdp « je suis visible ? » est bien crypté dans la trame. Oui ou Non ? Pourquoi ?

Non, le mot de passe n'est pas crypté. Parce que le mot de passe (mdp) a été de manière brute c'est-à-dire qu'il n'a pas été caché avant d'être posté.

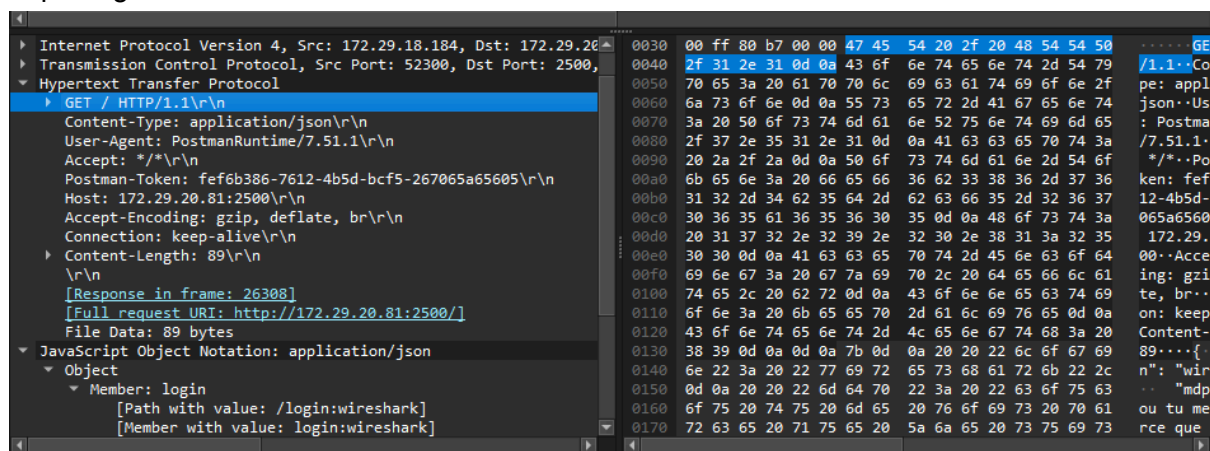
Partie 4 : analyse d'une requête http GET

6) Capture d'écran de la requête http GET dans wireshark

requête get dans postman

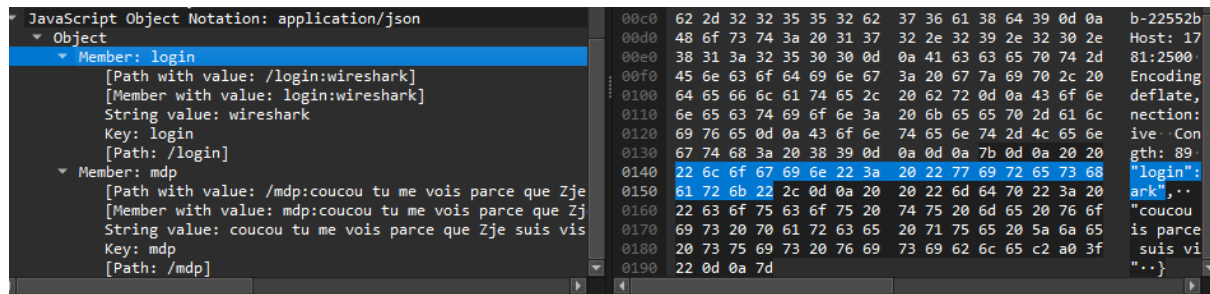


requête get dans wireshark

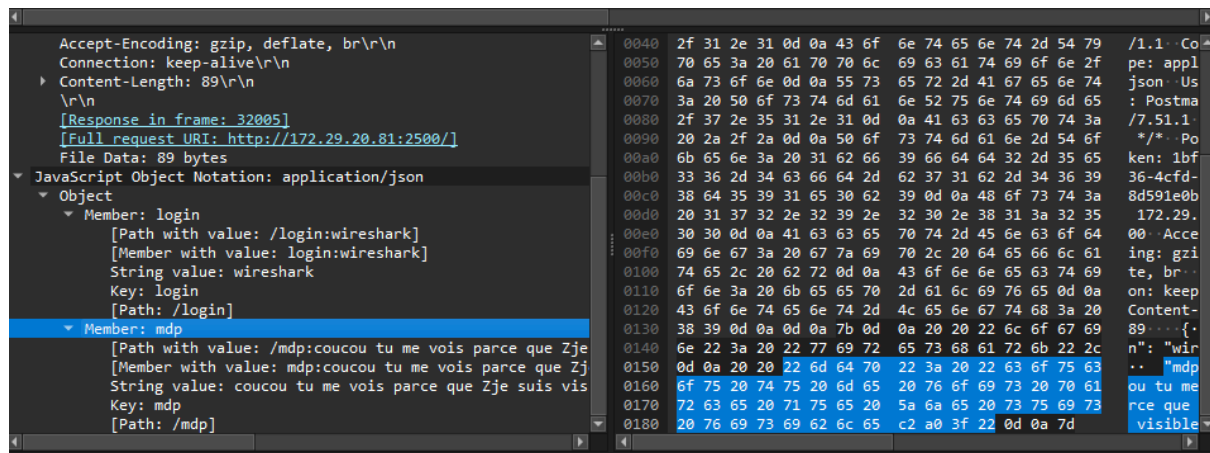


7) Capture d'écran de la requête http GET dans wireshark avec les params login et mdp

params login



params mdp



8) En Get est-ce que le mdp envoyé en paramètre est crypté ? faites une conclusion par rapport aussi au cryptage d'une requête POST

Non toujours pas car le chiffrement dépendant du https

9) Capture Au niveau du cryptage des paramètres et de la sécurité, quelle est la différence entre POST et GET ? Que proposez-vous comme solution ?

```
POST /test HTTP/1.1\r\n
Content-Type: application/json\r\n
User-Agent: PostmanRuntime/7.51.1\r\n
Accept: */*\r\n
Postman-Token: 037030ee-3f82-4ec4-9e1b-22552b76a8d9\r\n
Host: 172.29.20.81:2500\r\n
Accept-Encoding: gzip, deflate, br\r\n
Connection: keep-alive\r\n
Content-Length: 89\r\n
\r\n
```

```
GET / HTTP/1.1\r\n
Content-Type: application/json\r\n
User-Agent: PostmanRuntime/7.51.1\r\n
Accept: */*\r\n
Postman-Token: 1bf9fdd2-5e36-4cfd-b71b-4698d591e0b9\r\n
Host: 172.29.20.81:2500\r\n
Accept-Encoding: gzip, deflate, br\r\n
Connection: keep-alive\r\n
Content-Length: 89\r\n
\r\n
```


GET expose totalement les paramètres alors POST les masque partiellement.

Je propose d'utiliser un token ou même des cookies pour plus de sécurité ou on peut utiliser aussi le https.